

PROMPT IA 03

Développer le vertical slice complet Product -> Kafka -> Search -> OpenSearch -> Next.js

Objectif	Livrer un flux bout en bout réellement exécutable, testé et observable
Backend	Go - Product Service + Search Service
Messaging	Kafka + Protobuf + Apicurio + Outbox
Data	PostgreSQL + OpenSearch
Frontend	Next.js SSR/RSC utilisé comme BFF
Delivery	Docker + Tekton + Harbor + FluxCD + Flagger

Instruction centrale : ne génère pas un prototype jetable. Produis un incrément propre, maintenable, compilable, testé et conforme aux conventions de la plateforme. Travaille par petites étapes, exécute les tests après chaque étape, corrige immédiatement les erreurs, factorise le code partagé minimal et laisse le repository dans un état vert.

1. Rôle de l'IA

Tu agis comme un senior software engineer / platform engineer chargé d'implémenter le premier vertical slice de référence de la plateforme e-commerce. Tu dois écrire le code, les tests, les contrats, les migrations, les manifests et la documentation minimale nécessaires à un fonctionnement bout en bout.

- Ne pose une question que si une décision réellement bloquante manque. Sinon, choisis l'option la plus simple compatible avec les standards ci-dessous.
- Ne duplique aucune logique métier entre REST et gRPC : les transports appellent les mêmes use cases.
- N'introduis pas de nouvelle technologie non demandée.
- Toute décision critique et toute validation finale restent côté backend.

2. Résultat attendu

À la fin, ce scénario doit fonctionner de bout en bout :

```
Create Product
-> Product Service
-> PostgreSQL transaction
-> Outbox
-> Kafka product.updated.v1
-> Search consumer
-> OpenSearch projection
-> Search API
-> Next.js Customer Frontend
-> produit visible dans la page catalogue / recherche
```

Le flux doit être traçable avec correlation_id / trace_id, observable via OpenTelemetry et couvert par des tests automatisés.

3. Contraintes d'architecture obligatoires

- Backend standardisé en Go.
- REST/JSON pour l'exposition externe ; gRPC pour les appels synchrones inter-services.
- Protocol Buffers + Buf pour gRPC et les événements Kafka.
- Kafka via franz-go ; événements durables, idempotents, versionnés.
- Outbox Pattern obligatoire pour la publication fiable après commit PostgreSQL.
- PostgreSQL via pgx + sqlc ; aucune ORM.
- OpenSearch = read model reconstruisible ; PostgreSQL reste la source de vérité.
- OpenTelemetry pour traces/metrics/log correlation ; slog JSON pour les logs applicatifs.
- Docker multi-stage, image runtime distroless ou équivalent minimal validé.
- Secrets via OpenBao au runtime ; aucun secret dans Git ni dans le code.
- Kubernetes via FluxCD/GitOps ; aucun kubectl apply direct dans les scripts applicatifs.
- Tests unitaires + intégration + contrats + E2E.

4. Périmètre fonctionnel minimal

Product Service

- Créer un produit.
- Lire un produit par ID.
- Mettre à jour les champs de base utiles à la recherche.
- Gérer une version d'agrégat pour l'ordre/idempotence des événements.
- Publier un événement product.updated.v1 après commit via Outbox.

Search Service

- Consommer product.updated.v1.
- Dédupliquer par event_id.

- Ignorer proprement un événement plus ancien que la version déjà projetée.
- Indexer une projection Product dans OpenSearch.
- Exposer un endpoint de recherche simple par texte et filtres de base.

Frontend Next.js

- Page catalogue / recherche.
- Page produit minimale.
- SSR/RSC côté serveur ; le navigateur ne doit pas appeler directement les microservices internes.
- Next.js agit comme BFF de lecture et agrège uniquement ce qui est nécessaire à l'UI.

5. Structure de repository attendue

```
services/
product/
cmd/
internal/domain/
internal/application/
internal/infrastructure/
internal/transport/rest/
internal/transport/grpc/
api/openapi/
api/proto/
migrations/
tests/
search/
cmd/
internal/domain/
internal/application/
internal/infrastructure/
internal/transport/rest/
api/openapi/
api/proto/
tests/
web/
customer/
contracts/
proto/
platform/
deploy/
tekton/
flux/
go.work
```

Les packages communs doivent rester minimaux. Ne crée pas de framework interne générique si deux services n'en ont pas réellement besoin.

6. Modèle Product minimal

Utilise UUIDv7 pour les identifiants. Stocke les timestamps en UTC. Les champs peuvent rester volontairement minimaux pour ce premier slice :

```
Product
- id: UUIDv7
- sku: string
- name: string
- slug: string
- description: string
- status: ACTIVE | INACTIVE
- aggregate_version: int64
- created_at: timestampz
- updated_at: timestampz
```

Ne mets pas le stock effectif, les promotions, les taxes ou le prix calculé dans Product : ces responsabilités appartiennent à Inventory, Pricing et Tax.

7. Transactions et Outbox

Toute mutation Product doit écrire la donnée métier et l'événement Outbox dans la même transaction PostgreSQL.

```
BEGIN
INSERT/UPDATE product
INSERT outbox_event
COMMIT
```

Puis seulement :
Outbox publisher -> Kafka

- Aucun appel Kafka à l'intérieur de la transaction SQL.
- Le publisher doit être idempotent et reprendre après redémarrage.
- Prévoir statut/attempts/available_at/last_error ou un mécanisme équivalent propre.

8. Convention événementielle Kafka

Tous les événements utilisent l'enveloppe standard suivante :

```
event_id
event_type
aggregate_id
aggregate_version
occurred_at
producer
correlation_id
causation_id
trace_id
payload
```

- event_type = product.updated.v1
- partition key = aggregate_id
- consumer idempotent obligatoire
- compatibilité de schéma via Protobuf / Apicurio

9. Projection OpenSearch

Créer un index versionné et un alias. Ne jamais coder le nom physique dans le service de recherche.

```
products-read-v1
alias: products-read
```

Projection minimale suggérée :

```
id
sku
name
slug
description
status
aggregate_version
updated_at
```

- Mappings explicites ; pas de dynamic mapping libre.
- Le consumer Search refuse de régresser une projection si aggregate_version est inférieur ou égal à celle déjà indexée.
- Prévoir la possibilité de rebuild complet depuis les sources durables.

10. API REST et gRPC

REST expose uniquement les endpoints nécessaires au slice. gRPC doit réutiliser les mêmes use cases côté Product. La Search API peut rester REST pour l'usage frontend.

```
POST /api/v1/products
GET /api/v1/products/{id}
PATCH /api/v1/products/{id}
GET /api/v1/search?q=...&limit=20&cursor=...
```

- Erreurs structurées avec code stable + message + trace_id.

- Pagination par cursor pour Search.
- Validation stricte côté backend.

11. Frontend Next.js

- Utiliser App Router.
- Privilégier Server Components et fetch côté serveur.
- Ne pas dupliquer les règles métier.
- Afficher les états loading/error proprement.
- Supporter streaming/Suspense si utile sans complexité artificielle.
- Limiter le payload aux champs nécessaires.

Objectif de performance : une navigation publique doit déclencher le moins possible de round-trips backend réels. ATS/CDN et les caches seront traités côté plateforme ; le code frontend doit néanmoins être cache-friendly.

12. Observabilité

- Propager trace_id et correlation_id sur REST, gRPC et Kafka.
- Instrumenter les appels PostgreSQL, Kafka et OpenSearch via OpenTelemetry lorsque pertinent.
- Logs JSON structurés, jamais de secrets, JWT complets ou PII inutiles.
- Métriques minimales : request latency/error rate, outbox pending, Kafka consumer lag, projection errors, OpenSearch latency.

13. Sécurité

- Aucun secret dans Git, .env commité ou manifests Kubernetes.
- Prévoir les hooks/config pour OpenBao runtime injection.
- Backend authoritative : le frontend ne confirme jamais un état métier critique.
- Validation d'entrée à chaque frontière.
- readOnlyRootFilesystem, runAsNonRoot, allowPrivilegeEscalation=false, capabilities drop ALL, seccomp RuntimeDefault dans les manifests.
- NetworkPolicy/Cilium et politiques Istio fournies sous forme déclarative minimale.

14. Tests obligatoires

Unitaires

- use cases Product
- validation domaine
- mapping event
- consumer Search
- cursor/search parser

Intégration

- PostgreSQL via testcontainers-go
- Kafka si raisonnable via testcontainers
- OpenSearch via testcontainers
- Outbox publisher + reprise après erreur

Contract tests

- OpenAPI
- gRPC/Buf breaking checks
- Protobuf Kafka compatibility

E2E

Créer au moins un test E2E automatisé vérifiant :

1. POST Product
2. attente publication Kafka
3. attente projection OpenSearch
4. GET Search
5. le produit est présent
6. page Next.js affiche le produit

15. CI/CD attendu

```
PR
-> gofmt / go vet / golangci-lint
-> unit tests
-> integration tests
-> race detector
-> contract tests
-> frontend lint/typecheck/tests
-> build OCI
-> Trivy
-> Syft SBOM
-> Cosign
-> Harbor
-> FluxCD
-> Flagger canary
```

Ne produis pas de pipeline opaque. Les étapes doivent être lisibles et factorisées.

16. Ordre d'implémentation imposé

Étape 1 - Initialiser/compléter le monorepo et vérifier que tout compile.

Étape 2 - Créer le domaine Product + migrations PostgreSQL + sqlc.

Étape 3 - Implémenter les use cases et tests unitaires.

Étape 4 - Ajouter REST puis gRPC sur les mêmes use cases.

Étape 5 - Ajouter Outbox transactionnelle + publisher Kafka.

Étape 6 - Définir Protobuf/event envelope + tests de compatibilité.

Étape 7 - Créer Search consumer idempotent + OpenSearch index versionné.

Étape 8 - Créer Search API.

Étape 9 - Créer les pages Next.js catalogue/recherche/produit.

Étape 10 - Ajouter observabilité, manifests, CI/CD et E2E.

Étape 11 - Exécuter tous les tests, corriger, factoriser, documenter.

17. Règles de travail pour l'IA

- Avant de modifier un fichier, inspecte les fichiers liés et l'architecture existante.
- Réutilise les fonctions/helpers existants au lieu de dupliquer du code.
- Après chaque groupe de changements, exécute les tests concernés.
- Si un test échoue, corrige avant de poursuivre.
- Ne laisse pas de TODO critique non expliqué.
- Ne masque pas une erreur avec un mock inutile.
- Privilégie du code explicite et maintenable plutôt que des abstractions génériques prématurées.
- Les migrations doivent rester backward-compatible.
- Les nouvelles dépendances doivent être justifiées et minimales.
- À chaque étape, résume brièvement ce qui a été ajouté, testé et ce qui reste.

18. Definition of Done

- ☐ Le monorepo compile.

- ☐ Product peut être créé et relu.
- ☐ REST et gRPC utilisent les mêmes use cases.
- ☐ L'écriture Product + Outbox est atomique.
- ☐ Kafka reçoit product.updated.v1 avec l'enveloppe standard.
- ☐ Search consomme de façon idempotente et projette dans OpenSearch.
- ☐ Search API retrouve le produit.
- ☐ Next.js affiche le produit sans appel direct aux microservices depuis le navigateur.
- ☐ Tests unitaires/intégration/contrats/E2E verts.
- ☐ Race detector vert.
- ☐ OpenTelemetry/structured logging présents.
- ☐ Docker build réussi et image minimale.
- ☐ Manifests GitOps propres.
- ☐ Aucun secret commité.
- ☐ README indique comment lancer le slice localement et comment exécuter les tests.

19. Sortie finale attendue de l'IA

À la fin du développement, fournis un compte rendu concis comprenant :

- architecture réellement implémentée ;
- fichiers principaux créés/modifiés ;
- commandes de test exécutées et résultats ;
- contrats REST/gRPC/Kafka créés ;
- migrations appliquées ;
- limitations éventuelles restantes ;
- prochain incrément recommandé.

Ne considère pas le travail terminé tant que le scénario Create Product -> Kafka -> OpenSearch -> Search API -> Next.js n'est pas démontré par un test automatisé ou un script reproductible.